

CODEUP

Complete Flagship How-To Guide for Learners, Instructors, and Programs

PURPOSE

This guide takes CodeUp from first open to independent classroom use. Learners can write, run, understand, debug, and submit beginner Python. Instructors can create cohorts, assignments, guided learning, and support workflows. Programs can use the same accessibility-first workflow across a class or cohort.

About this guide

Item	Value
For	Learners, instructors, coordinators, program leads, and accessibility reviewers
Learner IDE	Open CodeUp IDE
Instructor sign-in	Open instructor sign-in
Source code	Open CodeUp repository
Covers	Python learning, accessibility settings, keyboard and optional voice workflows, Screen Reader Safe and CodeUp Voice speech modes, low-vision text sizing, nonvisual code orientation, precision reading, semantic code search, learning modes, classrooms, instructor tools, the complete learner command catalog, troubleshooting, and repository feature coverage.
Repository audit	Re-audited against CodeUp main, commit a45665f (7 September 2026), on 10 September 2026. User-facing command routes, first-party feature modules, templates, runtime paths, classroom surfaces, accessibility behavior, and regression tests were reviewed for this edition.

Item	Value
Guide revision	10 September 2026 - September accessibility, orientation, precision-reading, and output-navigation refresh.

Start here: choose your path

FLAGSHIP EDITION - SEPTEMBER 2026 REFRESH: Re-audited against the current main branch. This edition adds the two-mode speech architecture, XRCVC-driven accessibility changes, stronger indentation/orientation tools, semantic code search, Precision Mode, context resume, program-flow explanations, long-output orientation, and low-vision text sizing. Input examples keep the current short syntax: use Alexis and use 16.

CodeUp is easiest to learn by completing one complete workflow first, then using the detailed sections as reference. Use the document headings or your screen reader’s heading navigation to jump by section. Interface names are written as they appear in CodeUp, commands use monospace text, and every workflow is explained in words so the guide does not depend on screenshots.

If you are...	Start here
Learner	Part I. Begin with Sections 3-8. First goal: open CodeUp, write one short Python program, run it, and understand the output.
Instructor	Part II. Begin with Sections 44-53. First goal: create an instructor account, create a cohort, and publish one assignment.
Program coordinator	Part III. Begin with Section 69. First goal: prepare the instructor, devices, cohort, and first three beginner sessions.
Returning user	Part IV. Use the command reference, URLs, troubleshooting matrix, glossary, and quick-reference checklists.
Technical or accessibility reviewer	Part V. Use the repository coverage appendix to map CodeUp behavior to the first-party implementation and QA surfaces.

First 10 minutes: learner quickstart

The fastest way to understand CodeUp is to complete this short loop once. The detailed learner guide begins immediately after it.

1. Open **CodeUp IDE** and let the workspace finish loading.
2. Move to the Code editor and enter this program:

```
name = "Learner"
print("Hello", name)
```

3. Press Ctrl+Enter. Read or listen to Program output and confirm that the program prints “Hello Learner”.
4. Change the value of name, run the program again, and confirm that the output changes with the code.
5. Move to the command box and type “give me a code map”. Use the response to identify the structure of the short program.
6. Type “explain this code”. Compare the explanation with what you think each line does.

SUCCESS CHECK

You have completed CodeUp’s core learning loop: edit code → run → inspect output → change code → inspect structure → ask for an explanation. The rest of Part I expands each step.

Part I. Learner Guide

1. What CodeUp is

CodeUp is a browser-based, blind-first Python learning environment for beginners. A learner can type Python, run it, hear or read the output, understand errors, inspect program structure, follow values while code executes, work through lessons and guided projects, and participate in an instructor-managed classroom. The current build supports typed commands, keyboard workflows, optional microphone voice commands, Screen Reader Safe or CodeUp Voice narration, cursor-aware indentation explanations, fast structural overviews, exact code reading, context resume, semantic search, and output-orientation commands for nonvisual programming. CodeUp is designed for the early stage of learning Python, where concepts such as variables, indentation, loops, conditions, functions, program state, and errors still need to be understood. It can later serve as a bridge toward general development tools such as VS Code.

IMPORTANT Voice is optional. The complete learning workflow is available through the keyboard and screen-reader navigation. On a first visit, CodeUp defaults to Screen Reader Safe so an active screen reader remains the primary voice. Learners can switch to CodeUp Voice if they want CodeUp to narrate automatically. Microphone-based voice commands remain optional.

2. What a learner needs

Requirement	Recommended setup	Why it matters
Device	Laptop or desktop computer	The full IDE, keyboard workflow and classroom experience are designed for a desktop browser.
Browser	Chrome recommended	Chrome is recommended for the most consistent hosted IDE and optional microphone workflow. Keyboard and screen-reader workflows remain available throughout the IDE.
Internet	Stable connection	Needed for the hosted CodeUp application.
Screen reader	Learner's normal screen reader if used	CodeUp is designed to work alongside screen readers and Braille displays; it does not replace them.

Requirement	Recommended setup	Why it matters
Keyboard	Standard keyboard	Keyboard is the reliable baseline for editing, navigation, commands, and classroom tasks.
Microphone	Optional	Only required when the learner chooses voice recognition.
Headphones	Recommended in group settings	Useful when CodeUp Voice or other classroom audio is enabled, especially in group settings.

3. Opening CodeUp for the first time

1. Open **CodeUp IDE** in the browser.
2. Let the CodeUp workspace finish loading before beginning the lesson.
3. If the getting-started banner is shown, read it once. It introduces the main editor, command box, optional voice controls, and learning workflow.
4. If you use a screen reader, keep your normal screen reader running and select the matching assistive-technology profile in Settings when useful.
5. Start the first exercise with the editor and command box. Choose a speech mode in Settings: Screen Reader Safe keeps an active screen reader as the main voice, while CodeUp Voice makes CodeUp narrate automatically. Microphone-based voice commands remain optional.

4. The page structure

The main IDE is organized so keyboard and screen-reader users can move between major tasks without needing visual scanning.

Area	What it does	Typical learner action
Skip links	Jump directly to the editor or output	Useful immediately after page load with a keyboard or screen reader.
Header	Shows CodeUp, current mode and Settings	Change accessibility, display, speech mode, text size, and voice settings.
Learning tools	Snippets, project files and program inputs	Save reusable code, manage files, prepare input() values.
Editor	Main Python editing surface	Write, edit and review Python.

Area	What it does	Typical learner action
Output	Program output and runtime/error information	Read what the program printed or why it failed.
Command box	Typed natural-language command input	Run CodeUp commands without voice.
Voice controls	Optional microphone recognition and CodeUp narration	Speak commands when voice control is enabled; pause listening or stop CodeUp speech when needed.
More tools and Learning tools	Additional analysis, map, step, replay and learning controls	Use deeper accessibility and learning features when needed.
Classroom panel	Join and work inside an instructor cohort	View assignments, help state and classroom context.

5. Accessibility and display settings

5.1 Assistive-technology profile

Settings includes an assistive-technology profile. Choices include Default, NVDA, JAWS, Windows Narrator, VoiceOver, Orca, and a VS Code handoff-oriented profile. The profile helps CodeUp choose appropriate interaction and speech behavior. Choose the profile that matches the screen reader you actually use.

5.2 Speech mode: Screen Reader Safe

Screen Reader Safe is the default speech mode for a first visit. CodeUp stays quiet automatically so the learner's screen reader is the primary voice, while status updates are exposed through accessible ARIA regions. Explicit requests such as read output again or explain error can still speak because the learner asked for that narration. CodeUp supports the screen reader; it does not replace it.

5.3 Speech mode: CodeUp Voice

CodeUp Voice makes CodeUp the automatic narration channel. When this mode is active, CodeUp suppresses duplicate live-region announcements for content it is already speaking so CodeUp Voice and an active screen reader do not announce the same event at once. Use this mode when the learner wants CodeUp to narrate program output, guidance, and responses automatically.

5.4 Speech rate and voice

When CodeUp Voice is used, the speech settings provide a rate control from approximately 0.5x to 2x, a browser voice selector, and a Test Voice action. Choose a rate that is comfortable for learning. Faster speech is useful only when comprehension remains high.

5.5 Visual accessibility controls

Control	Purpose
Color vision: Standard	Default color presentation.
Protanopia	Alternative palette for red-weak/red-blind color vision.
Deuteranopia	Alternative palette for green-weak/green-blind color vision.
Tritanopia	Alternative palette for blue-yellow color-vision differences.
High Contrast	Increases contrast for low-vision use.
Dyslexia Mode	Adjusts text presentation for users who find that mode easier to read.
Reduce Motion	Reduces interface motion where supported.
Night Mode	Darkens the interface for users who prefer lower luminance.
Tab Leaves Editor	On by default: Tab and Shift+Tab move focus out of the code editor. Turn it off to make Tab indent code; Ctrl+] and Ctrl+[indent and outdent in both modes.
Text size: Normal / Large / Extra large	Scales the interface and Monaco editor for low-vision use. Large is approximately 125%; Extra large approximately 160%. The choice is persisted for later visits.

6. Keyboard basics

A learner can complete the essential workflow without voice. The browser and editor remain the main tools.

Key / action	Effect
Tab or Shift+Tab	Move through buttons, links, fields and disclosures.
Ctrl+Enter	Run the current Python program from the main editor.
Tab or Shift+Tab inside the editor	Leave the code editor forward or backward (the default), so keyboard users are never trapped in the editor. If you prefer Tab to indent code, turn off Tab Leaves Editor in CodeUp's display and accessibility settings.
Ctrl+] / Ctrl+[	Indent or outdent the current line of Python code. Works whether Tab Leaves Editor is on or off.

Key / action	Effect
Escape in Program inputs	Cancel the active input prompt.
Ctrl+M	Move focus out of the Monaco editor using CodeUp's editor escape path.
Arrow keys in Audio Blocks	Read/navigate blocks.
Ctrl+Arrow in Audio Blocks	Move a block.
[and] in Audio Blocks	Outdent or indent the current block.
Delete in Audio Blocks	Remove the selected block.
Ctrl+Enter in Audio Blocks	Compile blocks to Python.
Ctrl+Shift+Enter in Audio Blocks	Run compiled blocks.
Escape while CodeUp is speaking	Stop the current CodeUp speech.
Ctrl+Shift+M	Toggle microphone voice control.
Ctrl+Shift+P	Open the Command palette.
Alt+Shift+K	Open the full Keyboard shortcuts list.
Alt+Shift+O	Open accessibility and speech settings.

7. Writing Python manually

1. Move to the Code editor.
2. Type Python exactly as you would in a normal code editor.
3. Use indentation carefully. Python uses indentation as part of its syntax.
4. Run with Ctrl+Enter or the Run control.
5. Read the Program output area after the run.
6. If an error occurs, CodeUp exposes the error and its debugging tools instead of hiding the failure.

```
name = input("What is your name? ")
print("Hello", name)
```

For beginners, instructors should encourage manual typing for core exercises. Code generation can be useful for demonstrations or exploration, while manual typing provides stronger practice with syntax and structure.

8. Running a program and reading output

Run executes the current Python program inside CodeUp's sandboxed runtime. Printed output is shown in Program output. Runtime errors are available to the error-analysis and narration features. Output replay lets

a learner hear long output again without rerunning the program just to hear it a second time.

CLASSROOM TIP

For an exercise, ask learners to predict the output before pressing Run. This turns the run button into a comprehension check rather than a trial-and-error button.

8.1 Long output orientation

When a program prints many lines, orient yourself before replaying the whole output. These commands use the stored result of the most recent run, so you can quickly learn its size and endpoints without rerunning the program.

```
how many lines of output
first line of output
last line of output
```

Use read output or read last output when you want the complete stored output. Output orientation also handles output that contains blank lines rather than treating it as if the program never ran.

9. Python input() and program inputs

Programs that call input() need learner-supplied values. CodeUp supports both prepared input values and live input interactions.

9.1 Prepared input values

The Program inputs area can hold values before a run. The canonical command is use followed directly by the value. For example, use Alexis or use 16. Do not add “as input” to the command. If a program asks for more than one value, provide each value in prompt order.

```
read input prompt
use Alexis
use 16
read input values
clear input values
cancel input
```

INPUT COMMAND RULE: Type or say use followed directly by the value. Examples: use Alexis; use 16. Do not use “use 16 as input” in this guide.

9.2 Live input

Live input is useful when the program should ask for each value as it reaches input(). When CodeUp asks for a value, use the same short form: use Alexis, use 16, or use another value. For multiple prompts, answer one prompt at a time. Classroom commands such as Join ABC123 still route as classroom actions rather than program input.

10. The command box

The command box accepts natural-language instructions. Typed commands are the most reliable baseline because they do not depend on microphone permission or speech-recognition quality. Many important commands are deterministic and do not need an AI model.

10.1 A good beginner pattern

1. Write or open some code.
2. Type one short command.
3. Listen to or read CodeUp's response.
4. Check the editor/output to confirm what changed.
5. Use a follow-up command only after the first action is understood.

11. Optional voice commands

Voice commands use the browser microphone and speech-recognition controls. The command meaning is the same as typing the phrase into the command box. Voice recognition is optional and separate from speech mode: the microphone controls whether CodeUp listens, while Screen Reader Safe versus CodeUp Voice controls how responses are narrated.

```
start live assistant  
pause listening  
stop speaking
```

Ctrl+Shift+M toggles microphone voice control from the keyboard. The same deterministic command set remains available through typing if microphone recognition is unavailable or inconvenient.

Learners can switch between voice and the command box at any time, so the same workflow remains available through typing.

12. Generating beginner programs

CodeUp can create starter programs from natural-language requests. This is useful for demonstrations, examples, and exploration. In instructor-managed assignments, the instructor can disable generation while keeping explanations or other assistance available.

```
make a calculator  
make a marks average program  
make a loop program  
make a password checker  
make a program that asks for age
```

LEARNING RULE

For a real assessment or practice task, follow the instructor's AI policy. If generation is disabled, CodeUp will respect that classroom setting.

13. Editing code with natural-language commands

A learner can request focused edits instead of rewriting an entire program. CodeUp keeps enough context for common follow-up edits and can present changes for review.

```
make it use a function
add a loop
rename total to score_total
insert print Finished
comment this line
make this code better
```

After a substantial change, use the change-review tools before applying it blindly. Learning is stronger when the learner can explain why the code changed.

14. Reviewing changes before accepting them

CodeUp includes an Audio Diff-style workflow for comparing previous and proposed code. The learner can ask what changed, hear before-and-after information, ask for an explanation or risk assessment, then apply or reject the change.

```
what changed
read before and after
explain this change
is this risky
apply
reject
undo last change
```

WHY THIS MATTERS

A code assistant that immediately overwrites the learner's work can hide the learning process. Review/apply/reject keeps the learner involved in the decision.

15. Understanding program structure: Audio Code Map and project map

Code maps translate source structure into a form that can be explored without relying on visual indentation or minimaps. The map can identify where a program starts, major functions, classes, and blocks, and the learner's current structural location.

```
project map
give me a code map
where does the program start
where am I
read around me
```

The underlying structure tools parse Python syntax rather than simply reading each line aloud. This matters because code structure cannot be understood from spoken line text alone.

16. Variables and program state

State tools help a learner answer “what values exist right now?” instead of mentally simulating every line.

```
what variables exist
show program state
```

Variable/state views are most useful after a run or during a debugging walkthrough. In classroom assignments, instructors can allow or disable Variable Watch independently from other AI assistance.

17. Step Narration

Step Narration walks through execution in smaller steps and explains what the program is doing. It is useful for loops, changing variables, branch decisions and function calls.

step through this

A productive classroom pattern is to pause after each meaningful step and ask the learner what will happen next.

18. Errors and debugging

When Python fails, CodeUp surfaces the runtime error and supports several ways to investigate it.

```
explain error
where did it crash
what caused this
what value caused this
fix with explanation
read last error
```

18.1 Recommended learner debugging sequence

1. Run the program and listen to/read the actual error first.
2. Ask where the program crashed.
3. Ask what caused the error or what value was involved.
4. Try to predict the repair before asking CodeUp to fix anything.
5. If using “fix with explanation,” review the proposed change and understand the reason.
6. Run again and confirm the behavior rather than assuming the edit solved the problem.

19. Error Trace Narration and Mistake Replay

The runtime and accessibility layers keep structured information about recent failures. Error Trace Narration can explain the important location/cause information around a crash. Mistake or error replay lets

the learner revisit a recent failure and its correction rather than losing the teaching moment after the code changes.

20. Conditional breakpoints and focused debugging

CodeUp supports beginner-friendly debugging concepts, including breakpoints that can be associated with conditions such as a value crossing a threshold. These tools are intended to help a learner hear when a meaningful state is reached, rather than forcing continuous inspection of every line.

Example condition: `total > 10`

Use conditional breakpoints only after the learner understands the variable being watched and the condition being tested.

21. Indentation and structural cues

Python indentation is meaningful. CodeUp now provides cursor-aware, deterministic explanations of indentation and enclosing blocks so a blind learner does not have to infer hierarchy from spaces alone. These tools complement the Audio Code Map, Step Narration, sonification, and Audio Blocks views.

21.1 Why a line is indented

These commands explain the current cursor line using the actual Python block structure. They report enclosing loops, conditions, functions, and the current indentation depth instead of only reading leading spaces.

```
why is this line indented
what contains this line
how deep am I
what is inside this loop
```

If indentation is temporarily malformed, CodeUp can still explain the indentation problem rather than forcing the learner to repair it visually first.

21.2 Fast overview and structural navigation

Use overview for a quick semantic skim of the program. It reports the line count, named functions when present, and a useful structural landmark. Relative block navigation lets the learner move through the hierarchy without repeatedly searching for line numbers, and CodeUp speaks a short orientation cue after supported navigation jumps.

```
overview
parent block
first child
next sibling
previous sibling
```

21.3 Mental Map and unified execution explanation

Mental Map combines the useful context CodeUp already knows - current code location, recent execution/state information, a recent error, and the last recorded change when available - into a short orientation response. Unified execution questions combine trace and condition facts so the learner can ask one question instead of opening several tools.

```
mental map
what is my program doing
what just happened
why did that happen
```

21.4 Progressive disclosure

After a rich response such as an overview, Mental Map, semantic find result, or Error Trace, ask for more detail only when you need it. CodeUp stores that explanation context briefly and refuses to repeat stale details if the code has changed underneath it.

```
more
details
exact
```

21.5 Context resume

After opening another surface or moving away from the editor, Context Resume can restore the editor focus and speak a short cue about the last code location. This is especially useful after Audio Blocks, help, output review, or another analysis surface.

```
back to code
where was I
resume where I was
```

21.6 Precision Mode

Precision Mode is for exact proofreading on demand. Normal narration avoids reading every symbol because that becomes exhausting; Precision Mode can instead preserve case, punctuation, and character-level detail when a short line is difficult to verify nonvisually.

```
read exact line
read punctuation
spell current token
read character by character
```

21.7 Textual program flow and experimental Braille compact view

Textual program flow gives a conservative linear explanation of what the program does and points the learner back to Code Map or Step Narration when control flow becomes too complex to summarize safely. Braille Compact View is experimental: it compresses leading indentation into short depth markers such as Do, D1, or D3 so repeated spaces do not consume unnecessary Braille cells.

```
describe program flow
braille compact view
turn off braille compact view
```

21.8 Semantic find and confusable identifiers

Semantic find labels why each whole-word occurrence appears in the program - for example an assignment, condition, function call, return, string, or comment - instead of returning bare line numbers. If the file has a temporary syntax error during editing, CodeUp falls back to a conservative lexical scan rather than claiming the name is missing. For two similar identifiers, compare reads the exact character difference.

```
find total
compare userID and userId
more
```

22. Audio Blocks Mode

Audio Blocks is a block-based way to construct Python logic without depending on a visual drag-and-drop canvas. Blocks are numbered, keyboard-navigable, readable in order, and can be compiled into Python.

22.1 What learners can build

Audio Blocks covers output, variables, input, math, conditions, loops, lists, functions, comments, and basic program flow.

22.2 Common commands

```
open audio blocks
list block categories
add print block
add variable block
ask for age as number
read block order
compile blocks to Python
run blocks
transfer blocks to Python mode
switch to Python Code Mode
```

22.3 Keyboard workflow

Action	Result
Arrow keys	Move through/read blocks.
Ctrl+Arrow	Reorder the current block.
[or]	Outdent or indent the block.
Delete	Remove the current block.

Action	Result
Ctrl+Enter	Compile blocks into Python.
Ctrl+Shift+Enter	Run the block program.

Audio Blocks can be used as a bridge: build a small program structurally, compile it, then move into Python Code Mode to inspect the real source.

23. Python Code Mode

Python Code Mode is the main CodeUp workspace. Generation, natural-language editing, running, project files, error help, state watch, code maps, change review, classroom assignments and most advanced tools ultimately connect back to the Python editor.

24. Programming Literacy Mode

Programming Literacy Mode presents small learning missions instead of leaving a complete beginner in an empty editor. It is intended for concept-first learning.

```
start literacy mode
list lessons
start variables lesson
check lesson understanding
complete lesson
```

The learning system can combine a concept explanation, example, coding attempt, hints, challenge, and understanding check. The classroom curriculum uses a related staged structure.

25. Tutor Mode, hints and concept questions

```
start tutor mode
give me a hint
show fix
check my understanding
```

Hints should be used progressively. Start with a small clue, then ask for more only when needed. “Show fix” is the most direct intervention and is best reserved for when the learner has already attempted the task or the instructor permits it.

26. Asking questions about Python concepts

Concept Q&A is designed for questions such as “what is a dictionary?” or “why does this loop run three times?” In a classroom assignment, the instructor can allow concept questions while blocking full program generation.

Learners should distinguish between asking for an explanation and asking the system to perform the task. The former can support understanding while still leaving the learner responsible for the code.

27. Tutorial and onboarding

CodeUp includes a guided tutorial for new learners. The IDE can expose a Start Tutorial action and classroom learners who have not completed onboarding can be directed back to the short guided tutorial from their classroom home. For a first class or program session, use the tutorial before a graded or time-limited assignment. This separates learning the CodeUp interface from learning Python and makes later results easier to interpret.

28. Learner recaps, handoff packs and teacher reports

```
make codex handoff  
make a teacher report
```

Learning/reporting modules can summarize a learner's recent work or produce structured information for an instructor. These are supplementary reporting tools. Instructor classroom dashboards and cohort reports provide the official classroom-oriented view.

29. Snippets

The Learning tools area includes a snippet name field, a Save action and a list of saved snippets. Snippets are useful for small reusable patterns or examples. Instructors should avoid turning snippets into answer banks for assessments.

30. Multi-file projects

CodeUp supports project files so a learner can move beyond single-file examples. Project commands can list files, open a named file, create files, run a selected file and export the project.

```
read project files  
open helpers.py  
create file data.py  
run main.py
```

Project Map and structure tools are especially useful once multiple files exist because the learner can ask for the project structure rather than hunting visually through tabs or folders.

31. Exporting a project

```
export this project  
export for VS Code
```

Export packages the learner's project so it can be continued outside CodeUp. A VS Code-oriented export or handoff is useful when a learner is ready to move toward a professional coding environment.

32. Joining an instructor classroom from the IDE

The main IDE contains a Classroom panel. A learner can join with a cohort code supplied by the instructor. The join flow works through the interface or through deterministic classroom commands.

32.1 Manual join

1. Open the main IDE.
2. Open Classroom.
3. Enter the instructor-provided class code.
4. Enter the learner name.
5. Activate Join.
6. After joining, the Classroom area changes from the join form to the learner's cohort summary.

32.2 Typed/voice join

Join ABC123

CodeUp can then ask for the learner's name if it is not already known. Joining through the command system is deterministic and does not require Groq AI.

33. What joining a cohort changes

After joining, CodeUp remembers the learner's membership in the browser using a learner token cookie. Classroom assignments, course progress, projects, and help status can appear while the learner continues to use the same main IDE. Leaving or switching learners clears local classroom cookies; it does not delete the instructor's stored classroom records.

34. Learner classroom home

The dedicated learner classroom page shows the learner's current work.

Section	Information and actions
Assignments	Title, current status, due date and Open in editor.
Course	Start the built-in course or continue where the learner left off.
Onboarding	Link back to the short guided tutorial if onboarding is incomplete.
Guided projects	Project title and completed checkpoints.
Help	Open requests, instructor-helping status, cancel request and new help request.
Leave or switch learner	Clears this browser's classroom identity so another learner can join.

35. Opening an assignment

When a published assignment is opened, CodeUp brings its title, instructions, and starter code into the learner workflow. The active assignment becomes part of the classroom context, so questions such as “what should I do?” can return its instructions.

```
what should I do
what am I supposed to do
what do I need to do
repeat the assignment
repeat the instructions
read this assignment
```

36. Working on and submitting an assignment

- 1. Open the assignment.
- 2. Read the title and instructions fully.
- 3. Write the code manually unless the assignment instructions or policy explicitly allow a different workflow.
- 4. Run the program and inspect output/errors.
- 5. Use permitted explanations, hints, code maps, Step Narration, or Variable Watch as needed.
- 6. Submit only when the program represents the learner’s intended answer.
- 7. After submission, use Back to CodeUp or Back to IDE to return to the normal editor without losing classroom membership.

```
submit
submit my assignment
submit this assignment
turn this in
back to ide
back to codeup
```

37. Classroom AI and tool permissions from the learner perspective

An instructor can configure what assistance is available for each assignment. Accessibility fundamentals remain available regardless of these settings. Screen-reader access, keyboard navigation, output access, focus management, and program input are never disabled by the assignment AI policy.

Capability	What it controls
Generate	Full code generation from a request.
Fix	Automatic/proposed repair behavior.
Explain	Code explanation.

Capability	What it controls
Hint	Hint generation.
Error help	Explanations around runtime errors.
Concept Q&A	Questions about Python concepts.
Audio Code Map	Structural code map assistance.
Step Narration	Step-by-step execution explanation.
Variable Watch	Following selected values/program state.

```

what can AI help me with
what can AI do
what is allowed

```

38. Built-in Python Foundations course

Python Foundations contains twelve modules. Each module uses a staged learning sequence with concept material, examples, a coding attempt, hints, a challenge, and a quiz or understanding check where applicable.

#	Module	Main idea
1	Printing	Producing output.
2	Variables	Storing and changing values.
3	Input	Receiving user data.
4	Data types	Understanding common value types.
5	Conditions	Making decisions.
6	For loops	Repeating over a sequence/range.
7	While loops	Repeating while a condition remains true.
8	Lists	Ordered collections.
9	Dictionaries	Key-value collections.
10	Functions	Reusable named behavior.
11	Debugging	Finding and understanding mistakes.
12	Mini project	Combining learned concepts.

39. Built-in guided project: Student Marks Program

The built-in Student Marks Program guides a learner through a small complete program. Its checkpoints cover creating a marks dictionary, calculating a total, calculating an average, and printing the result. Checkpoint verification is deterministic from the learner's code rather than being judged only by a language model.

40. Custom guided projects

Instructors can add custom guided projects. Learners see checkpoint progress and can open a project in the IDE. Custom checkpoints can check for printing, function calls, named assignments, selected Python keywords, and arithmetic operators.

Checkpoint type	What an instructor can check
contains_print	Learner included print output.
contains_call	Learner called a specified function.
contains_assignment_named	Learner assigned to a named variable.
contains_keyword	Learner used for, while, if, def, try or class as configured.
contains_operator	Learner used +, -, *, /, // or % as configured.

41. Asking an instructor for help

A learner can request instructor help from the classroom page or with a classroom command.

```
I need help
ask teacher for help
ask instructor for help
is my instructor helping
cancel help
```

The learner may include an optional message. When an instructor marks the request as “helping,” the learner can see that the instructor is reviewing it. The learner can cancel an open request if it is no longer needed.

42. Leaving or switching learner

Use Leave or switch learner when a shared computer needs to move to another student. This clears the browser's classroom identity. It does not erase the learner's stored submissions or instructor records.

Part II. Instructor Guide

43. Instructor responsibilities

An instructor's first job is to make the classroom predictable: create the cohort, share the join code, publish clear work, choose the level of assistance allowed, watch learner progress, respond to help requests, and review submissions. The instructor dashboard is organized around those repeated classroom tasks.

44. Creating an instructor account

1. Open the instructor sign-in page. [instructor sign-in page](#)
2. In Create an instructor account, enter your name.
3. Choose a username.
4. Choose a password of at least 8 characters.
5. Activate Create account.
6. After account creation, sign in from the same page if needed.

INSTRUCTOR ACCOUNT PRACTICE

Use separate instructor accounts for staff who manage classes. Keep credentials private and store them using your organization's normal account-management practice.

45. Signing in and out

The instructor login page provides Username and Password fields. After sign-in, the Instructor dashboard shows the instructor's cohorts. Use Sign out when leaving a shared or public computer.

46. Creating a cohort

1. On the Instructor dashboard, locate Create a cohort.
2. Enter a descriptive cohort name, for example "Python Beginners - Cohort A."
3. Activate Create cohort.
4. The cohort appears in Your cohorts with its join code, learner count, status, and actions.

If the cohort name is missing, CodeUp shows an accessible server-side error, associates it with the field, preserves the typed value, and focuses the invalid field.

47. Join codes

Every active cohort receives a short unique join code. Give this code only to the learners who should join that cohort. Learners can type it into the Classroom join form or use "Join CODE" in the command box.

DO NOT TREAT THE JOIN CODE AS A PASSWORD

The join code identifies a cohort. Instructor authentication remains separate. Do not use a learner join code as a substitute for staff credentials.

48. Archiving and reactivating a cohort

Active cohorts can be archived from the Instructor dashboard and archived cohorts can be reactivated. An archived cohort should not accept new joins through the normal active-code lookup. Archive cohorts that are finished rather than deleting historical learner work.

49. Opening a cohort dashboard

Select the cohort name from Your cohorts. The cohort dashboard is the instructor's main operational screen.

Dashboard area	What it shows and controls
Cohort at a glance	Learners enrolled, active learners, lessons completed, assignments submitted, guided projects completed, programs run/successful, help requests, course completions.
Cohort details	Join code and cohort rename.
Learners	Each learner, live status, recent activity and progress totals.
Create an assignment	Title, instructions, starter code, dates, expected concepts, policy and assessment flag.
Assignments	Assignment status, AI policy and due date, with links to details.
Header actions	Help queue, custom lessons, custom projects, report and All cohorts.

50. Understanding learner live status

The instructor dashboard calculates a simple learner status from classroom records. Examples include instructor helping, help requested, offline, submitted, running code, and working. Use the status as classroom context, not as a score.

51. Near-live instructor updates

While a cohort dashboard is open, CodeUp polls a lightweight live summary. The learner table, assignment table, and help count can update without a full page refresh. Updates are applied without rebuilding the whole table, which helps keyboard and screen-reader focus stay stable.

Meaningful events such as a learner joining, requesting help, or submitting an assignment can be announced once through the instructor’s screen-reader status region. Routine polling stays quiet.

52. Creating an assignment

Field	Meaning and recommendation
Title	Required. Make it short and specific.
Instructions	Explain the goal, required behavior and any restrictions.
Starter code	Optional code the learner receives when opening the task.
Start	Optional date/time when work should begin.
Due date	Optional deadline shown to learners.
End or close	Optional closing time.
Expected concepts	Comma-separated concepts such as dictionaries, variables, print output.
AI policy preset	Starting set of assistance permissions.
This is an assessment	Marks the assignment as an assessment and supports stricter permission workflows.

52.1 Example beginner assignment

```
Title: Student Marks Program
Instructions:
Create a Python program that stores marks for at least three subjects in a dictionary.
Calculate the average of all marks and print the final average.
Expected concepts: dictionaries, variables, arithmetic, print output
```

53. Draft versus published assignments

A newly created assignment begins as a draft. Learners should not receive it as normal published work until the instructor opens the assignment and activates Publish assignment. This gives the instructor time to review instructions and permissions first.

54. Assignment schedule

The assignment detail page allows the instructor to save an optional start time, due date and end/close time. Use these fields only when the dates matter to the teaching plan. A short introductory course can operate without rigid deadlines.

55. AI policy presets

A preset provides a starting combination of the nine assignment capabilities. The instructor can then fine-tune individual permissions.

Preset	Use case
FULL	All assignment assistance capabilities enabled.
EXPLANATIONS_ONLY	Focus on explanation while restricting stronger intervention such as complete generation.
HINTS_ONLY	Give hints with tighter restrictions on direct answers.
ERROR_HELP_ONLY	Allow error-oriented support while keeping other assistance restricted.
ASSESSMENT	Strict assessment-oriented starting point.
OFF	Disable the classroom-gated assistance capabilities.

ACCESSIBILITY STAYS AVAILABLE

Screen-reader access, keyboard navigation, output access, focus management, and program input remain available even when an instructor turns AI or learning-assistance capabilities off.

56. Fine-tuning the nine assignment capabilities

Capability	Instructor decision question
Generate	Should the learner be allowed to request complete or substantial generated code?
Fix	Should CodeUp be allowed to propose/directly support fixes?
Explain	May the learner ask for explanations of their code?
Hint	May the learner request hints?
Error help	May the learner receive assistance understanding runtime errors?
Concept Q&A	May the learner ask Python-concept questions?
Audio Code Map	May the learner use the structural code-map tool?
Step Narration	May the learner use execution-step narration?

Capability	Instructor decision question
Variable Watch	May the learner inspect variable/program state through this tool?

56.1 Suggested settings by activity

Activity	Suggested starting approach
First demonstration	FULL or EXPLANATIONS_ONLY so the instructor can show capabilities.
Guided practice	Explain + Hint + Error help + Concept Q&A + accessibility analysis tools as appropriate.
Independent practice	Reduce Generate/Fix first. Keep explanations/error support if the learning goal allows.
Assessment	Start from ASSESSMENT or OFF and explicitly enable only what the assessment rules permit.

57. Locking permissions for an assessment

Assignment permissions can be locked. Once locked, the permission controls are disabled until an instructor deliberately unlocks them. This reduces accidental policy changes after an assessment begins.

58. Duplicating an assignment

The assignment detail page can duplicate an existing assignment. A checkbox can duplicate it as a strict assessment starting with the stricter assessment-oriented configuration. Duplication is useful when reusing a task across cohorts or creating a practice and assessment version.

59. Monitoring submissions

The Assignment details page contains a submission table for every learner. It shows status, run count, successful run count, submission time, and an Inspect link when code is available.

Run counts are context, not a quality score. A beginner may need many runs while learning. Review submitted code and learning progress rather than ranking learners by how few attempts they used.

60. Inspecting a learner submission

Use Inspect from the assignment or learner detail view to open the saved/submitted code associated with that learner and assignment. This is useful when responding to a help request or reviewing a submission after class.

61. Learner detail page

Selecting a learner opens a detailed instructor view.

Section	What it contains
Course: Python Foundations	Current module, completed module count, onboarding status and last curriculum activity.
Module table	Per-module status, quiz score and Reset action.
Assignments	Status, run counts and Inspect link.
Concept progress	Recorded concept state.
Recent activity	Recent classroom/progress event types.
Full report	Generated report text for the learner.

62. Resetting a learner module

The instructor can reset a module from the learner detail page. Use this when a learner needs to repeat a learning sequence or when an instructor needs a clean retry. Explain the reset to the learner first so progress does not appear to disappear unexpectedly.

63. Help queue

The Help queue is designed for real classroom triage. Open requests are shown longest-waiting first.

Queue state	Instructor actions
Open requests	See learner, assignment, message and waiting time. Mark as helping or mark resolved.
Currently helping	See active cases. Add an optional resolution note and mark resolved.
Recently resolved	Review recent completed help requests and notes.

When an instructor marks a request as helping, the learner can see that the instructor is reviewing it. This reduces repeated help requests caused by uncertainty.

64. Custom lessons

An instructor can create lessons that learners use through the same lesson interface as the built-in course.

Field	Use
Title	Required lesson name.

Field	Use
Learning objective	What the learner should understand or be able to do.
Explanation	Concept explanation.
Starter code	Optional initial code.
Instructions	What the learner should build/change.
Expected concepts	Comma-separated concepts.
Optional challenge	Extra task after the main attempt.
Optional expected output	Reference only. It is explicitly not auto-graded.
Quiz question	Optional comprehension check.
Choices	One choice per line.
Correct choice number	Zero-based index: 0 means the first line.

CONTENT DESIGN

Keep custom lessons small. One clear concept and one meaningful coding action is usually better for a beginner than a page full of instructions.

65. Custom guided projects

Custom guided projects let an instructor define a larger task with deterministic checkpoints based on the learner's real code.

Field	Use
Title	Required project name.
Instructions	Overall project goal.
Starter code	Optional starting point.
Expected concepts	Concepts expected in the project.
Checkpoint title	Human-readable step name. Leave blank to skip that checkpoint row.
Checkpoint type	Deterministic code condition.
Check detail	Variable name, keyword, operator or comma-separated names depending on type.

The instructor template provides five checkpoint rows. Checkpoints use deterministic verification based on the learner's code structure and content rather than a free-form AI opinion.

66. Cohort reports and CSV export

The cohort dashboard provides a Report action. The reporting layer aggregates classroom progress and can provide downloadable CSV data for instructor review. Treat exported learner data as educational records and store it according to your organization's privacy practices.

67. AI service status

The Instructor dashboard includes an authenticated AI service status link. Use it when an optional AI-dependent feature appears unavailable. Core classroom records, deterministic curriculum and project checks, Python execution, and accessibility fundamentals are separate from the Groq service.

68. Instructor accessibility

Instructor pages are built for keyboard and screen-reader use. They use a skip link, clear heading structure, a main landmark, native form controls, labeled fields, accessible tables, centralized status and alert regions, and focus-aware validation. Live updates are designed to preserve focused rows and typed form values.

Part III. Running a Class or Program

69. Recommended first-cohort setup

For a first program rollout, start with a cohort size the instructor can comfortably support. Use one clear lead instructor or coordinator and a short sequence of beginner Python sessions. Scale according to staffing, devices, and learner support needs. The goal is a repeatable routine: join, learn, run, debug, ask for help, submit, and review progress.

Implementation element	Recommended starting point
Learners	A manageable first cohort based on available staff, devices, and learner support needs.
Instructor/coordinator	One named person who owns the cohort and consolidates feedback.
Sessions	A short sequence of beginner sessions, paced to learner progress.
Devices	Each learner has a laptop/desktop, browser, internet and usual assistive technology.
Teaching level	Complete beginners are acceptable.
Primary goal	Confirm learners can independently enter, run, understand and debug beginner Python while the instructor can manage the class.

70. Suggested Session 1: Orientation, output and variables

1. Open CodeUp and complete the short interface orientation/tutorial.
2. Confirm that keyboard and screen-reader navigation work comfortably for each learner.
3. Write and run `print("Hello")`.
4. Introduce variables and print variable values.
5. Demonstrate the command box with “what can I do here” and one explanation command.
6. Show how CodeUp reads an error by deliberately introducing one small syntax or name error.
7. End with a two-variable mini exercise and collect immediate accessibility feedback.

71. Suggested Session 2: Input, conditions and debugging

1. Review variables.
2. Introduce `input()` and CodeUp program-input handling.
3. Write a simple age, marks, or name program.

4. Introduce if/else.
5. Use the error trace or explanation tools after a deliberate mistake.
6. Show Audio Code Map or Step Narration on a short conditional program.
7. Have learners ask for instructor help through the classroom system once so the workflow is familiar before they truly need it.

72. Suggested Session 3: Assignment and guided project

1. Instructor publishes a short assignment.
2. Learners open it from the classroom and write code manually.
3. Learners run, correct mistakes and submit.
4. Instructor reviews incoming submissions and help requests and inspects one submission.
5. Open the Student Marks guided project or an instructor-created project.
6. Finish with learner feedback about navigation, speech, errors, instructions, and instructor support.

73. Roles during implementation

Role	Responsibility
Program coordinator	Chooses learners, schedules sessions, identifies instructor, consolidates program feedback.
Instructor	Creates cohort/assignments, teaches Python, manages help queue, reviews learner progress.
Learners	Use CodeUp normally and report accessibility/learning friction honestly.
CodeUp maintainer / support contact	Handles product support, deployment support, and improvements based on classroom feedback.
Accessibility reviewer	Supports screen-reader and assistive-technology review where useful for the learner group.

74. What to review during the first cohort

The first cohort should focus on learner independence, classroom flow, accessibility comfort, and beginner Python understanding. These observations help the instructor decide what to reinforce in later sessions.

Area	Useful evidence
Independent operation	Could the learner open the IDE, write/run code, read output and recover from an error with decreasing assistance?

Area	Useful evidence
Classroom workflow	Could learners join, open work, submit and request help? Could the instructor see and act on those events?
Accessibility friction	Any focus loss, duplicate speech, unclear labels, heading/landmark problems, keyboard traps or speech collisions.
Learning	Could learners explain the variable/condition/loop/function they used?
Session consistency	App responsiveness, successful program runs, classroom synchronization, assignment flow and help response.
Feedback	Short learner and instructor comments about what helped learning and which instructions could be clearer.

75. Data and privacy

Classroom records include instructor accounts, cohorts, learner display names, learner tokens, assignments, code and progress, help requests, curriculum progress, guided-project progress, concept progress, and activity events. Instructor exports can contain learner progress. Treat these as classroom records.

- Use learner display names that fit your organization’s privacy policy. The learning workflow does not require full legal names.
- Do not share instructor passwords.
- Keep cohort join codes within the intended learner group.
- Do not put secrets, passwords, or sensitive personal data inside programming exercises or submitted code.
- Store downloaded reports and CSV files only where your organization normally stores learner records.

76. Accessibility feedback and support

Instructors and coordinators can use learner feedback to refine classroom routines for different screen-reader, browser, keyboard, and speech preferences. The most useful feedback describes the exact action, what the learner heard or experienced, and what would have made the step clearer.

When reporting an accessibility issue, include the screen reader or other assistive technology, browser, exact action, what was announced or displayed, and what the learner expected. This makes support and follow-up faster.

77. Screen Reader Safe versus CodeUp Voice

CodeUp now exposes two speech modes instead of separate speech toggles. Screen Reader Safe keeps automatic CodeUp narration quiet so the learner’s screen reader is the primary voice. CodeUp Voice makes

CodeUp narrate automatically and suppresses duplicate live-region announcements for content it is already speaking. If a learner reports overlapping speech, first confirm the selected speech mode in Accessibility and speech settings.

78. AI settings and classroom continuity

AI assistance is optional and instructor-controlled. Core classroom controls, deterministic command routing, curriculum checks, guided-project checkpoints, Python execution, keyboard navigation, and accessibility structure operate independently from optional AI assistance.

79. Classroom support procedure

Problem	Immediate classroom response	Follow-up information
Voice recognition fails	Use typed commands and keyboard controls.	Browser, microphone permission, exact phrase.
CodeUp speech conflicts with the learner's screen reader	Set Speech mode to Screen Reader Safe so the learner's screen reader is the primary narration channel.	Selected speech mode, screen reader, and browser combination.
Program error	Read actual error, use error help if permitted, ask instructor if needed.	Code and error text, excluding sensitive data.
Accessibility question	Use the learner's normal keyboard or assistive-technology workflow and note the exact step.	Assistive technology, browser, action, and what would make the step clearer.

Part IV. Complete Operational Reference

80. Core command reference

Category	Example phrases
Run and output	run / read output / read last output / show run history / expected output [text]
Input	read input prompt / use Alexis / use 16 / read input values / clear input values / cancel input
Generate and edit	make a calculator / make it use a function / add comments / simplify this code / rename variable total to score_total
Read and navigate	read my code / go to line 5 / read function greet / go to definition of total / file outline / read comments
Structure and maps	give me a code map / project map / what is inside the loop / what comes after the loop / list functions
State and execution	show program state / what is score now / watch variable score / step through this / run with step narration
Debug	explain error / where did it crash / what value caused this / read full traceback / fix with explanation
Breakpoints	set breakpoint at line 8 / pause when total is greater than 10 / why did it pause / continue execution
Review changes	what changed / read before and after / is this risky / next change / previous change / apply this change / undo last change
Projects and snippets	read project files / create file data.py / run main.py / save this as snippet / list snippets / export this project
Learning	start literacy mode / start tutor mode / give me a small hint / quiz me on this code / start tutorial / start error practice
Audio Blocks	open audio blocks / list block categories / read block order / run blocks / transfer blocks to Python mode
Quality and safety	preflight check / check indentation / check brackets / check strings / check beginner style / sandbox check

Category	Example phrases
Data accessibility	preview csv / summarize csv / list csv columns / read csv row 3 / describe chart / sonify data
Voice and help	pause listening / resume / repeat / help / all commands / start live assistant

CANONICAL COMMANDS Use the short canonical forms below first. CodeUp recognizes many aliases and repairs some speech-recognition near misses, but a classroom guide should teach one dependable phrase for each action. For Python input values, always use use <value>, such as use Alexis or use 16.

80.1 Run, output, repetition and help

Canonical command	What it does	Notes / useful forms
run	Run the current program.	Also: run code, run program, execute, try it.
read output	Read the stored program output.	Use read output again for replay.
read last output	Replay the most recent output.	Also: repeat output; what did it print.
expected output Hello	Compare actual output with an expected value.	Also: compare output to [text]; should print [text].
show run history	Summarize recent runs.	Also: what have I run; run summary.
reset run state	Clear recent run/output state.	Also: clear last output; clear run history.
repeat	Repeat the last spoken response.	Also: again; say that again.
help	Show context-sensitive help.	Also: show commands; what can I say; guide me.
all commands	Show the larger command help surface.	Also: full help; list commands.
how many lines of output	Report the number of lines in the stored output.	Useful before replaying a long result.
first line of output	Read the first stored output line.	Quick orientation at the start of a result.

Canonical command	What it does	Notes / useful forms
last line of output	Read the last stored output line.	Quick end-state check without replaying everything.

80.2 Python input() commands

Canonical command	What it does	Notes / useful forms
read input prompt	Repeat the current Python input prompt.	Use before supplying a value if the prompt was missed.
use Alexis	Supply a text value.	Canonical syntax: use followed directly by the value.
use 16	Supply a numeric value.	Do not say “use 16 as input”.
read input values	Read prepared/pending values.	Useful before a run with several input() calls.
clear input values	Clear prepared values.	Also: clear inputs; reset inputs.
cancel input	Cancel the active input interaction.	Useful if the learner wants to stop the waiting prompt.
live input mode	Use interactive input prompting.	Also: interactive input; live run.
preflight input mode	Use prepared/batch inputs.	Also: preflight inputs; batch input mode.

80.3 Generate, insert and edit code

Canonical command	What it does	Notes / useful forms
make a calculator	Generate a beginner calculator.	Starter generation.
make a marks average program	Generate a marks-average example.	Starter generation.
make a program that asks for age	Generate an input() example.	Starter generation.

Canonical command	What it does	Notes / useful forms
loop from 1 to 5	Insert a deterministic counting loop.	Also: print numbers 1 to 5.
loop even to 10	Insert an even-number loop.	Also: print even numbers up to 10.
make a safe while loop	Insert a bounded while-loop example.	Unsafe infinite-loop wording triggers clarification.
make a variable example for marks	Insert a variable example.	Deterministic beginner template.
make an if statement for age	Insert a conditional example.	Deterministic beginner template.
make a list example	Insert a list example.	Deterministic beginner template.
make a function example	Insert a function example.	Deterministic beginner template.
insert function greet	Add a named function.	Also: add function greet; create function greet.
insert class Student	Add a named class.	Also: add class Student; create class Student.
add parameter name to function greet	Add a parameter.	Short form add param is accepted.
insert a for loop	Add a for loop.	Can specify the iterable in natural language.
add an if statement for score > 50	Add a conditional.	Condition follows the command.
add a while loop while count < 5	Add a while loop.	Keep the condition bounded.
add variable score with value 0	Create a variable assignment.	Natural language assignment form.
insert print Finished	Append a print line.	Also: insert print hello.
insert "print(score)" at line 5	Insert code at a specific line.	Line-aware deterministic edit.
replace line 5 with print(total)	Replace a specific line.	Also: change line 5 to ...

Canonical command	What it does	Notes / useful forms
delete line 5	Delete a specific line.	Line-aware deterministic edit.
comment this line	Comment the current line.	Also: comment current line.
uncomment this line	Remove the comment marker.	Current-line edit.
duplicate this line	Copy the current line below.	Also: copy this line below.
delete blank lines	Clean blank lines.	Also: remove blank lines.
add comments	Add beginner-readable comments.	Useful after code is working.
simplify this code	Simplify common beginner code shapes.	Review the resulting diff.
change it to while loop	Transform a loop form.	Review the resulting diff.
rename variable total to score_total	Rename a symbol safely.	Also: change name from total to score_total.
make it use a function	Refactor current code into a function.	Use change review afterward.
suggest next line	Ask for a next-line suggestion.	Also: what should I write next.
choose option 1	Choose one offered suggestion.	Also: select option 1; pick option 1.
clear editor	Clear the current editor.	Also: reset editor; clear code.

80.4 Read, explain and navigate code

Canonical command	What it does	Notes / useful forms
read my code	Read the whole current program.	Also: read my program; read line by line.
narrate file	Read/narrate the whole file.	Useful for non-visual review.
walk me through this code	Explain the program in sequence.	Also: explain what this code does.

Canonical command	What it does	Notes / useful forms
analyze code	Analyze the current code.	Also: review code; check code.
analyze deeper	Request more detail.	Also: line by line; more detail.
summarize code	Give a concise program summary.	Also: summary of this file.
read line 5	Read a specific line.	Line number can be spoken.
describe line 5	Describe what is on a line.	Useful before an edit.
explain this line	Explain the current line.	Cursor-aware deterministic surface.
read around me	Read nearby lines.	Also: give me context.
go to line 5	Move to a line.	Also: jump to line 5.
read function greet	Read a named function.	Function-aware navigation.
go to definition of greet	Jump to a definition.	Also: where is greet defined.
where is score used	Find references.	Also: find uses of score.
file outline	Read file structure.	Also: outline this file; read file outline.
list functions	List defined functions.	Also: show my functions.
go to the function	Go to the first/appropriate function.	Meaning-based navigation.
go to the loop	Go to a loop.	Meaning-based navigation.
go to the print statement	Go to a print statement.	Meaning-based navigation.
read current block	Read the current structural block.	Also: describe current block.
next function	Move to the next function.	Previous function is also available.

Canonical command	What it does	Notes / useful forms
next class	Move to the next class.	Previous class is also available.
next block	Move to the next block.	Context-aware block navigation.
what file am I in	Report the active project file.	Works in single-file and project contexts.
what does this file do	Summarize the file role.	Project-aware.
go to main function	Jump to main().	Also: find main function.
open file with main function	Open the file that owns main.	Role-based file navigation.
open the file that handles score	Open a file by likely role.	Project-aware.
read comments	Read comments in the file.	Also: list comments.
jump to changed line	Navigate to the most recent changed line.	Also: where was the last change.
where am I	Report cursor/execution context.	Also: where is my cursor.
read exact line	Read the current line with exact case and symbol detail.	Precision Mode; also: read this line exactly; read this exactly.
read punctuation	Read only the punctuation on the current line, in order.	Precision Mode for symbol checking.
spell current token	Spell the current identifier, keyword, or number.	Also: spell this token; read current token.
read character by character	Read every character on the current line one at a time.	Most granular Precision Mode reading; very long lines are capped.
find total	Find whole-word occurrences and label why each occurrence is present.	Replace total with the term; use more for the expanded list.
compare userID and userId	Compare two identifiers character by character.	Useful for case differences and names that sound almost identical.

Canonical command	What it does	Notes / useful forms
back to code	Return focus to the editor and restore the previous code context.	Also: where was I; resume where I was.
more	Expand the last rich explanation or search result.	Also: details; exact. Stale stored explanations are rejected after code changes.

80.5 Structure, maps and code shape

Canonical command	What it does	Notes / useful forms
give me a code map	Describe AST-based code structure.	Also: code map; map this code.
project map	Describe project entry point, files and relationships.	Also: what does each file do.
where does the program start	Identify the project/program entry point.	Also: what is the entry point.
what imports what	Read the import graph.	Project Map surface.
what functions are in this project	List project functions.	Project Map surface.
what is inside the loop	Describe loop contents.	Audio Code Map sub-query.
what comes after the loop	Describe code after a loop.	Audio Code Map sub-query.
how nested am I	Report nesting depth.	Also: deepest nesting depth.
show structure	Open/show structural summary.	Also: show code structure.
read outline	Read the code outline.	Structure navigation.
sonify function greet	Sonify a function structure.	Audio structural cue.
sonify class Student	Sonify a class structure.	Audio structural cue.
sonify block	Sonify the current block/line.	Also: play the code.

Canonical command	What it does	Notes / useful forms
sonify file	Sonify whole-file shape/indent profile.	Audio structural cue.
overview	Give a fast structural skim of the current program.	Line count, named functions when present, and a key landmark.
why is this line indented	Explain why the current cursor line is nested.	Cursor-aware enclosing-block explanation.
what contains this line	Read the enclosing block chain for the current line.	Also: what am I inside.
how deep am I	Report indentation depth at the current line.	Current cursor depth, not whole-file maximum nesting.
parent block	Move to the current block's parent.	Also: go up one block.
first child	Move into the first child block.	Also: go inside this block.
next sibling	Move to the next block at the same structural level.	Previous sibling is also supported.
describe program flow	Read a conservative textual program-flow explanation.	Also: explain the flow; how does this program work.
mental map	Summarize the most useful current code and execution context.	Useful after navigation, a run, an error, or a recent change.
braille compact view	Show an experimental compact indentation view for Braille displays.	Uses D-depth markers; say turn off braille compact view to stop.

80.6 Code checks, readability and sandbox safety

Canonical command	What it does	Notes / useful forms
preflight check	Check likely issues before running.	Also: will this run.

Canonical command	What it does	Notes / useful forms
check indentation	Explain indentation problems.	Also: where is indentation wrong.
check brackets	Check balanced brackets/parentheses.	Also: are my brackets balanced.
check strings	Check quotes/string closure.	Also: are my strings closed.
check long lines	Find long/readability-heavy lines.	Also: readability check.
check names	Find naming conflicts/shadowing.	Also: check for shadowing.
show code stats	Summarize code counts/statistics.	Also: code statistics.
show nesting depth	Report code nesting.	Also: check nesting.
show todos	Find TODO comments.	Also: list todos.
list imports	Read imports in current code.	Also: check imports.
show requirements	List project requirements/packages.	Also: what packages does this project need.
check missing files	Check project imports for missing local files.	Also: check project imports.
sandbox check	Check for risky/unsafe code patterns.	Also: find risky code.
explain blocked import	Explain import policy.	Also: why is [module] blocked.
show safe imports	List allowed/safe imports.	Also: what imports are allowed.
check project	Run project health checks.	Also: is my project ready.
read file tree	Summarize project files.	Also: summarize project files.
check beginner style	Run beginner style checks.	Learning-oriented style feedback.

Canonical command	What it does	Notes / useful forms
check readable names	Check name readability.	Learning-oriented style feedback.
check function length	Check oversized functions.	Learning-oriented style feedback.
check too much nesting	Check excessive nesting.	Learning-oriented style feedback.
check confusing names	Check confusing identifiers.	Learning-oriented style feedback.
explain style issues	Explain detected style issues.	Use show more style issues for more.

80.7 Program state, Variable Watch and Step Narration

Canonical command	What it does	Notes / useful forms
what variables exist	List variables detected in the program.	Also: summarize variables.
show program state	Trace and narrate current state.	Uses sandboxed execution.
what is score now	Read a specific traced value.	Replace score with the variable name.
watch variable score	Focus on a variable.	Also: track score; monitor score.
read watched variables	Read current watch list/values.	Useful after a traced run.
unwatch score	Stop watching one variable.	Also: stop watching score.
clear watched variables	Remove all watches.	Also: stop watching all variables.
step through this	Start step-by-step tracing.	Also: start stepping; trace this code.
next step	Move forward one narrated step.	Context-aware.
previous step	Move backward one narrated step.	Context-aware.

Canonical command	What it does	Notes / useful forms
explain current step	Explain the current trace step.	Also: explain this step.
go to first step	Jump to first trace step.	Useful in review.
go to last step	Jump to last trace step.	Useful in review.
repeat step	Repeat the current step narration.	Useful with screen readers/speech.
explain loop state	Explain iterations and loop value state.	Also: what is the loop doing.
why did this condition pass	Explain a true condition from trace facts.	Use explicit word condition.
why did this condition fail	Explain a false condition from trace facts.	Use explicit word condition.
what did the program print	Summarize traced program output.	Also: what was printed.
run with step narration	Run and narrate execution changes.	Also: run and narrate.
read variable values	Read traced variable values.	Also: show variable values.
what changed in this step	Explain state changes at current step.	Step-aware.
only announce changes	Reduce narration to changed state.	Also: changes only.
execution story	Narrate the run as an execution story.	Also: tell execution story.
what is my program doing	Give a unified explanation of current execution facts.	Combines trace, state, and condition information when available.
what just happened	Explain the most recent relevant execution step or run context.	Also: why did that happen; what happened on that step.

80.8 Breakpoints and focused debugging

Canonical command	What it does	Notes / useful forms
set breakpoint at line 8	Set a line breakpoint.	Also: pause at line 8.
pause when total is greater than 10	Set a conditional audio breakpoint.	Condition is evaluated against traced state.
list audio breakpoints	Read conditional breakpoints.	Also: show breakpoints.
why did it pause	Explain the verified pause reason.	Also: explain current pause.
remove breakpoint on line 8	Remove one line breakpoint.	Line-specific.
clear breakpoints	Remove all breakpoints.	Also: remove all breakpoints.
disable breakpoints	Temporarily disable breakpoints.	Keeps configuration.
enable breakpoints	Re-enable breakpoints.	Restores configured breakpoint behavior.
continue execution	Continue after a pause.	Also: resume execution.
step into	Enter a function during debug stepping.	Also: inside the function.
step out	Leave the current function during debug stepping.	Also: exit the function.

80.9 Error Trace Narration and Mistake Replay

Canonical command	What it does	Notes / useful forms
explain error	Narrate the recent error.	Also: trace error; narrate error.
read errors only	Give a concise error summary.	Avoids reading the full traceback.
where did it crash	Report crash file/line.	Uses recent runtime error.
what caused this	Explain likely cause.	Error-aware.

Canonical command	What it does	Notes / useful forms
what value caused this	Read the exact value when trace evidence supports it.	Falls back safely if unknown.
read full traceback	Read the full stored traceback.	Use only when needed.
what should I test next	Suggest the next debugging test.	Keeps debugging learner-led.
fix with explanation	Propose a repair with teaching explanation.	Does not silently apply the proposal.
replay mistake	Compare a recent failure and correction.	Mistake Replay.
why does the fixed version work	Explain why corrected code works.	Uses error/replay history.
show changed lines	Read only the changed lines.	Useful after a correction.
explain output diff	Explain why output changed between runs.	Also: why did this run differently.

80.10 Change review, Audio Diff and undo

Canonical command	What it does	Notes / useful forms
what changed	Review the latest code diff.	Also: show what changed; audio diff.
read before and after	Read old and new code around the change.	Also: show before and after.
explain this change	Explain the selected change.	Review before accepting.
is this risky	Read change risk classification.	Also: how risky is this change.
next change	Move to next change.	Diff navigation.
previous change	Move to previous change.	Also: prev change.
accept this change	Accept/keep the selected change.	Review workflow.

Canonical command	What it does	Notes / useful forms
accept all changes	Accept every pending change.	Use cautiously.
reject all changes	Reject every pending change.	Use cautiously.
apply this change	Apply a pending proposal/fix.	Also: apply the fix.
undo last change	Restore the previous code state.	Also: reject this change.
compare before and after	Compare change states.	Change/mistake review surface.

80.11 Projects, files, snippets, macros and bookmarks

Canonical command	What it does	Notes / useful forms
read project files	List project files.	Also: file tree; what files are in this project.
open helpers.py	Open a project file.	Also: switch to helpers.py.
create file data.py	Create a file.	Also: add new file data.py.
rename file data.py to scores.py	Rename a project file.	Project-aware.
delete file scores.py	Delete a project file.	Confirmation may be required by UI.
run main.py	Run a selected project file.	Also: execute main.py.
explain project structure	Explain file/project organization.	Also: describe project structure.
explain requirements	Explain project requirements.	Project-aware.
export this project	Package the project.	For continuing outside CodeUp.
export for VS Code	Prepare a VS Code-oriented handoff/export.	Professional transition path.
save this as snippet	Save current code as a snippet.	Can auto-name.

Canonical command	What it does	Notes / useful forms
save this as snippet called greeting	Save with a name.	Named snippet.
list snippets	List saved snippets.	Also: show my snippets.
load snippet greeting	Load a snippet.	Named snippet.
preview snippet greeting	Read a snippet without loading it.	Also: peek at snippet greeting.
rename snippet old to new	Rename a saved snippet.	Snippet management.
remember this as quick run	Save a macro.	Also: save this as macro quick run.
use macro quick run	Run/load a macro.	Also: run macro quick run.
list macros	List saved macros.	Macro management.
share macro	Create a shared macro reference where available.	Sharing surface.
use shared macro ABCD	Load a shared macro by code.	Four-character share code.
bookmark this	Bookmark output/state.	Can optionally name the bookmark.
read from bookmark	Return to/read a bookmark.	Can specify a bookmark name.
list bookmarks	List saved bookmarks.	Output navigation.

80.12 Learning, Tutor Mode, practice and tutorial

Canonical command	What it does	Notes / useful forms
start literacy mode	Open Programming Literacy Mode.	Structured beginner missions.
list lessons	List available literacy lessons.	Also: show lessons.
start variables lesson	Start a named lesson.	Replace variables with lesson name.
next lesson	Move forward in learning flow.	Previous lesson also available.
lesson status	Read current lesson status.	Literacy Mode.

Canonical command	What it does	Notes / useful forms
what am I learning	Explain the current learning goal.	Literacy Mode.
give me lesson starter code	Load/offer lesson starter code.	Literacy Mode.
check lesson understanding	Run the lesson understanding check.	Learning evidence.
complete lesson	Mark/complete the lesson flow when requirements are met.	Literacy Mode.
start tutor mode	Turn on Tutor Mode.	Also: turn on tutor mode.
tutor mode status	Report Tutor Mode state.	Tutor surface.
give me a hint	Give a hint.	Tutor Mode canonical phrase.
give me a small hint	Give the smallest hint.	Progressive hint ladder.
give me a bigger hint	Escalate hint strength.	Progressive hint ladder.
why is this hint useful	Explain the learning purpose of a hint.	Hint pedagogy.
show solution steps	Show structured solution steps.	More direct help.
show fix	Show a direct fix when policy allows.	Most direct tutor intervention.
let me try again	Return control to the learner.	Tutor Mode.
check my understanding	Ask/score an understanding check.	Also: quiz me on this code.
ask me a question	Generate an understanding question.	Learning check.
what mistake did I make	Reflect on a recent attempt.	Understanding surface.
give me a similar exercise	Create related practice.	Also: make practice question.
grade my attempt	Assess the learner attempt.	Learning surface.

Canonical command	What it does	Notes / useful forms
quiz me	Start a quiz/challenge.	Can add a topic.
bug challenge	Start a debugging challenge.	Practice surface.
start tutorial	Open onboarding tutorial.	Also: open tutorial.
restart tutorial	Restart onboarding.	Tutorial state.
tutorial next	Advance tutorial step.	Tutorial state.
skip tutorial	Exit/skip tutorial.	Also: close tutorial.
practice variables	Open tutorial practice for a topic.	Also supports print, if, for, while.
start error practice	Start error-practice mode.	Accessible learning pack.
practice indentation errors	Practice a specific error type.	Also name/type/syntax errors.
read error challenge	Read current error challenge.	Error practice.
check error fix	Check the learner repair.	Error practice.
give error hint	Get a progressive debugging hint.	Error practice.
show error solution	Show solution when needed.	Error practice.
next error challenge	Move to next challenge.	Error practice.
exit error practice	Leave error-practice mode.	Error practice.
make codex handoff	Create a structured handoff pack.	Also: prepare codex handoff.
make a teacher report	Create the session/learner teacher report.	Reporting surface.

80.13 Accessible learning path and teacher tools

Canonical command	What it does	Notes / useful forms
start learning path	Start the accessible learning path.	Also: continue learning path; reset learning path.

Canonical command	What it does	Notes / useful forms
where am I in the learning path	Read learning-path position.	Accessible learning.
repeat lesson	Repeat the current path lesson.	Also: skip lesson.
show lesson goal	Read the goal.	Also: check lesson; give lesson hint.
show keyboard shortcuts	Read keyboard shortcuts.	Also: practice keyboard shortcuts.
enter navigation mode	Enable learning navigation mode.	Also: navigation mode on.
what navigation mode am I in	Report navigation mode state.	Accessible learning.
next symbol	Navigate learning/code symbols.	Also: previous symbol.
next loop	Navigate loops.	Also: previous loop.
next error	Navigate errors.	Also: previous error.
next todo	Navigate TODOs.	Also: previous todo.
read current scope	Read the current code scope.	Accessible navigation.
teacher mode on	Enable teacher-oriented learning/report surface.	Also: teacher mode off.
generate lesson report	Generate lesson reporting.	Accessible learning teacher mode.
generate student report	Generate student reporting.	Teacher mode.
generate mistakes report	Generate mistakes reporting.	Teacher mode.
show common mistakes	Summarize common mistakes.	Teacher mode.
export teacher report	Export the teacher report.	Teacher mode.
reset teacher report	Reset teacher-report state.	Teacher mode.
include code in teacher report	Include code in teacher report.	Also: exclude code from teacher report.

80.14 Audio Blocks complete command surface

Canonical command	What it does	Notes / useful forms
open audio blocks	Enter Audio Blocks Mode.	Also: switch to audio blocks.
what mode am I in	Report current mode.	Audio Blocks/Python mode.
list block categories	List available block types.	Also: what blocks can I add.
list output blocks	List blocks in a category.	Also import, variable, math, condition, loop, list, function, exception, input, comment.
read block workspace	Read the block workspace.	Also: summarize blocks.
read block 3	Read a specific block.	Also: read selected block.
select block 3	Select a block.	First block / last block are supported.
where am I in blocks	Report block position.	Audio Blocks navigation.
read nested blocks	Read nesting.	Also: read children of block 3.
move block 3 up	Reorder a block.	Down/before/after are supported.
indent block 3	Nest a block.	Outdent also supported.
delete block 3	Delete a block.	Audio Blocks edit.
put block 3 inside block 2	Nest by command.	Can target else of a block.
remove block 3 from loop	Remove block nesting.	Audio Blocks edit.
undo block change	Undo block edit.	Redo block change also supported.
clear block workspace	Clear blocks.	Audio Blocks reset.
duplicate selected block	Duplicate selection.	Can move selected block up/down.

Canonical command	What it does	Notes / useful forms
add print text Hello	Create a print-text block.	Block creation.
add print variable score	Create a variable-output block.	Block creation.
set variable score to 0	Create/update a variable block.	Block editing.
append 5 to numbers	Append to a list variable.	Block editing.
edit block 3	Edit a block.	Then set its field/value.
set block 3 condition to score > 50	Edit a condition.	Field-aware block editing.
rename block variable score to total	Rename block variable references.	Block edit.
compile blocks to Python	Compile blocks.	Also: convert blocks to code.
preview generated code	Read compiled Python before transfer.	Audio Blocks review.
run blocks	Compile/run the block program.	Audio Blocks execution.
explain generated code	Explain compiled Python.	Bridge to source code.
compare blocks and code	Compare block model and Python.	Learning bridge.
transfer blocks to Python mode	Send compiled Python to editor.	Also: copy blocks to Python mode.
convert code to blocks	Try importing compatible Python.	Also: import code into blocks.
explain why code cannot become blocks	Explain unsupported conversion.	Safe fallback.
start block lesson	Start Audio Blocks lesson.	Next/check/hint/solution/exit commands supported.
export block project	Export blocks and generated Python.	Also: download block project; export blocks and python.

Canonical command	What it does	Notes / useful forms
switch to Python mode	Return to Python Code Mode.	Also: open python mode.

80.15 CSV, data accessibility and sonification

Canonical command	What it does	Notes / useful forms
preview csv	Preview a CSV file.	Can name a .csv file.
summarize csv	Summarize CSV shape/content.	Also: describe csv.
list csv columns	Read column names.	Data accessibility.
read csv row 3	Read a selected row.	Data accessibility.
highest value in score	Find the highest value in a column.	Use the real column name.
lowest value in score	Find the lowest value in a column.	Use the real column name.
average of score	Calculate/read column average.	Use the real column name.
compare columns maths and science	Compare two columns.	Data accessibility.
describe chart	Describe a chart as text.	Also: make chart accessible; read chart as text.
sonify data	Sonify data.	Audio data representation.
sonify column score	Sonify one column.	Use stop sonification to stop.

80.16 Voice, display, mentoring and product guidance

Canonical command	What it does	Notes / useful forms
pause listening	Pause voice recognition.	Also: voice off; pause voice.
resume	Resume voice recognition.	Also: start listening; voice on.
set high contrast mode	Set a visual color/contrast mode.	Also protanopia, deuteranopia, tritanopia, standard.

Canonical command	What it does	Notes / useful forms
start mentor mode	Start mentor/learning mode.	Also: teach me.
mentor stop	Stop mentor mode.	Mentor control.
tiny hint	Request a minimal mentor hint.	Also: don't give me the answer.
bigger hint	Request a stronger mentor hint.	Mentor hint ladder.
exact fix	Request direct mentor fix.	Most direct help.
slow walkthrough	Ask for slower explanation.	Also: explain like I'm new.
did I fix it	Check progress after an edit.	Also: is it better now.
say that shorter	Shorten mentor explanation.	Also: explain simpler.
beginner mode	Set mentor explanation level.	Intermediate mode also supported.
use shorter answers	Set concise mentor preference.	Session preference.
map of my code	Ask mentor for code map.	Also: summarize my code structure.
why not use VS Code	Explain CodeUp's role relative to VS Code.	Product guidance.
how is CodeUp different from Codex	Explain role difference.	Copilot comparison also supported.
what happens after CodeUp	Explain professional transition.	Also: how do students move to VS Code.
open accessible coding tools	Open/show accessible coding tools.	Learning/pathway surface.
explain Quorum	Explain Quorum in the accessibility pathway.	Also: how is CodeUp different from Quorum.
explain VS Code handoff	Explain transition/handoff.	Accessible pathway.

Canonical command	What it does	Notes / useful forms
show accessible coding pathway	Show the broader pathway.	Transition guidance.
intel toolkit status	Show optional Intel integration status.	Aliases: intel status; show intel optimization report.

81. Classroom command reference

Classroom category	Representative phrases
Navigation	go to classroom go to editor go to code editor go to command box go to command input go to output go to assignments go to lessons go to course go to projects go to help go to join classroom
Back	back to ide return to ide go back to ide back to codeup return to codeup
Join and identity	join [CODE] join classroom join my class enter class code what class am I in leave class
Assignments	open my assignments how many assignments do I have what is new what is due what is overdue open assignment 1 continue my assignment read this assignment repeat the instructions what am I supposed to do what do I need to do what can AI help me with what is allowed submit submit this assignment turn this in
Course	start my course continue my course restart this module next previous read this lesson show the example start quiz course progress
Projects	open my projects how many projects do I have continue my project what is my current step repeat project instructions I finished this step check project progress next checkpoint project hint
Help	I need help ask teacher for help ask instructor for help cancel help is my instructor helping
Orientation	what should I do now repeat orientation what can I say how does classroom mode work

The classroom command layer is deterministic. Unambiguous classroom phrases such as Join CODE are matched before generic AI routing.

82. Main URLs

Destination	Link	Who uses it
Learner IDE	<u>Open learner IDE</u>	Learners and demonstrations.
Instructor login / registration	<u>Open instructor sign-in</u>	Instructors.
Instructor dashboard	<u>Open instructor dashboard</u>	Signed-in instructors.
Learner join fallback	<u>Open learner join page</u>	Learners who are not joining inside the IDE.
Learner classroom home	<u>Open learner classroom home</u>	Joined learners.
AI service status	<u>Open AI service status</u>	Signed-in instructors checking optional AI service status.
Storage status	<u>Open storage status</u>	Signed-in instructors checking the active classroom storage backend and health.
Repository	<u>Open CodeUp repository</u>	Technical review and source code.

83. Troubleshooting matrix

Symptom	Likely cause	Action
Nothing happens when I speak	Microphone permission, unsupported browser speech-recognition API, recognition not active	Type the same command. Check Chrome microphone permission. Voice is optional.
CodeUp and my screen reader talk over each other	Speech mode does not match the learner's preferred narration channel	Open accessibility and speech settings and choose Screen Reader Safe when a screen reader should be the primary voice. Choose CodeUp Voice only when CodeUp should narrate automatically.

Symptom	Likely cause	Action
A command was misunderstood	Long/ambiguous wording or recognition error	Use a shorter deterministic phrase or type it.
Program waits for input	Python input() reached	Use the short value command, for example use Alexis or use 16. For multiple prompts, answer them in order. Use read input prompt if you need the prompt repeated.
Program output is long	Automatic speech may summarize very long output.	Use how many lines of output, first line of output, or last line of output to orient yourself. Use read output or read last output for the complete stored result.
Assignment not visible	Draft/not published, schedule, stale learner context	Instructor confirms published status; learner syncs/reopens classroom.
Submit unavailable/rejected	No active assignment or task state	Open the assignment first and confirm current classroom.
Instructor does not see a learner immediately	Near-live poll interval	Wait several seconds or refocus the instructor tab; do not repeatedly recreate learner.
Help request seems unanswered	Instructor has not marked helping	Check help status; instructor opens Help queue.
Navigation or focus feels unexpected	Browser, assistive-technology, or dynamic-page interaction	Record the exact control and action, then share it with the instructor or CodeUp maintainer for support.
I lost my place after opening another tool	Focus/context moved away from the editor	Say where was I or back to code to restore the editor context and hear a short orientation cue.

Symptom	Likely cause	Action
I cannot tell why a line is indented	Nested block context is unclear	Say why is this line indented, what contains this line, or how deep am I.
Two variable names sound almost the same	Identifiers differ only by case or a few characters	Say compare name1 and name2. For the current line, use spell current token or read character by character.

84. Glossary

Term	Meaning in CodeUp
Audio Code Map	Accessible structural description/navigation of code.
Audio Blocks	Keyboard/audio-oriented block programming mode that compiles to Python.
CodeUp Voice	Speech mode in which CodeUp narrates responses automatically and avoids duplicate live-region announcements for content it is already speaking.
Classroom	Instructor/learner cohort system layered into the main IDE.
Cohort	An instructor-managed class/batch with a join code.
Command box	Text field for natural-language CodeUp commands.
Concept progress	Recorded evidence and state for Python concepts.
Guided project	Larger learner task with deterministic checkpoints.
Live assistant	Optional microphone/listening workflow for voice commands.
Programming Literacy Mode	Concept-first learning mode for beginners.
Screen Reader Safe	Default speech mode that keeps automatic CodeUp narration quiet so the learner's screen reader is the primary voice.
Step Narration	Step-by-step execution explanation.
Variable Watch and state watch	Tools for tracking program values and state.
Mental Map	Short composed orientation using current code location and recent run/error/change context when available.

Term	Meaning in CodeUp
Precision Mode	On-demand exact line, punctuation, token spelling, and character-by-character reading for nonvisual proofreading.
Progressive Disclosure	The more/details/exact follow-up system for expanding a recent rich explanation without repeating everything by default.
Context Resume	Restores editor focus and the learner's previous code location after moving to another surface.
Braille Compact View	Experimental view that represents leading indentation with compact D-depth markers for Braille displays.
Semantic Find	Search that labels the role of code occurrences instead of returning bare line numbers, with a safe lexical fallback for temporarily broken syntax.

85. Learner quick-reference checklist

- Open the CodeUp IDE and confirm the editor and Program output are reachable.
- Use keyboard first; voice is optional.
- Select the correct screen reader profile if applicable.
- Write code manually for instructor exercises unless the assignment policy says otherwise.
- Run with Ctrl+Enter or Run.
- Read the error before asking for a fix.
- Use code map, state watch, or Step Narration when they help you understand the program.
- In a classroom, open the assignment before submitting.
- Use “I need help” when instructor assistance is required.
- Use Back to CodeUp after submitting to continue in the normal IDE.

86. Instructor quick-reference checklist

- Sign in with your own instructor account.
- Create one cohort per learner group and share its join code.
- Create assignments as drafts, review them, then publish.
- Choose AI permissions intentionally. Accessibility fundamentals remain available.
- Keep the cohort dashboard open during a live class for learner/help/submission updates.
- Use Help queue to acknowledge requests.
- Inspect submissions and learner detail when intervention is needed.
- Use custom lessons/projects only when they add teaching value.
- Handle exported reports and CSV files carefully because they contain learner records.

- Confirm the instructor account, cohort, learner join flow, sample assignment, help queue, and submission flow before the first session.

86.1 User-facing function index

This index is the product-level definition of “functions” for the guide: learner, instructor, accessibility, runtime, project, learning, reporting, and classroom capabilities a user can actually invoke or encounter. Private Python helper functions are implementation details and are mapped by module in Part V rather than presented as learner commands.

Function family	Included capability
Editing	Monaco Python editor, line edits, natural-language edits, safe rename, comments, simplify/transform, undo.
Execution	Sandboxed Python run, output capture, output line-count/first-line/last-line orientation, long-output replay, expected-output comparison, run history.
Python input()	Prepared values, live prompts, prompt replay, clear/cancel, sequential multi-input handling with use <value>.
Audio Code Map	AST-derived structure, hierarchical nesting, inside/after-loop queries, functions/classes, current cursor context, and structural landmarks.
Project Map	Files, entry point, imports, functions, file roles and project structure.
Meaning navigation	Go to line/function/class/loop/print/error/main, definitions/references, comments, changed line, active file, plus short orientation cues after supported jumps.
Structural sonification	Block/function/class/file shape and indentation-oriented audio cues.
Variable Watch	List variables, trace values, watch/unwatch variables, read watched values.
Step Narration	Trace execution, next/previous/repeat/first/last step, explain current step, changes-only mode.
Conditional audio breakpoints	Line/condition breakpoints, pause reason, enable/disable, continue, step in/out.
Error Trace Narration	Crash location, cause, value evidence, concise error, full traceback, next-test guidance.

Function family	Included capability
Mistake Replay	Replay prior error/correction, changed lines, why fixed version works.
Audio Diff review	Before/after, risk, explanation, next/previous change, apply/accept/reject/undo.
Audio Blocks	Keyboard/audio block creation, editing, nesting, lessons, compile/run, Python transfer, export.
Programming Literacy Mode	Structured lessons, status, starter code, practice, understanding checks, completion.
Tutor and mentor	Progressive hints, explanation-first, retries, direct fix when allowed, preference controls.
Tutorial/onboarding	Start/restart/next/skip, topic practice, interface orientation.
Error practice	Error challenges by type, hints, fix checks, solutions and progression.
Accessible learning path	Learning path navigation, block/Parsons practice, navigation mode, scope and keyboard practice.
Concept Q&A	Beginner explanations and questions about Python concepts/current code.
Understanding checks	Quiz on current code, questions, similar exercise, grade attempt, mistake reflection.
Snippets	Save, name, list, load, preview and rename reusable code snippets.
Macros and bookmarks	Save/use/list/share macros; bookmark/list/revisit output points.
Multi-file projects	Create/open/rename/delete/run files, project structure, requirements and health.
Export and handoff	Project export, VS Code-oriented export, Codex handoff pack, block project export.
Code quality checks	Preflight, indentation, brackets, strings, long lines, naming, style, nesting, TODOs.
Sandbox/import safety	Risky-code checks, blocked-import explanation, safe-import list.

Function family	Included capability
CSV/data accessibility	CSV preview/summary/rows/columns, highest/lowest/average/compare, chart text, sonification.
Accessibility settings	AT profile, Screen Reader Safe or CodeUp Voice, speech rate/voice, Normal/Large/Extra large text, contrast/color vision, dyslexia, motion, and night mode.
Voice interaction	Start/pause/resume listening, stop speaking, typed-command fallback, recognition repair.
Classroom membership	Join/leave/switch learner, learner token/context, classroom home.
Assignments	Open/read instructions, AI capability policy, work/run, submit, return to IDE.
Python Foundations	12-module built-in curriculum with attempts/challenges/quizzes.
Guided projects	Built-in Student Marks project plus instructor-defined deterministic checkpoints.
Help queue	Learner help request/cancel/status; instructor open/helping/resolved workflow.
Instructor cohorts	Create/archive/reactivate cohorts, join codes, learner live status.
Instructor assignments	Draft/publish/schedule/duplicate, presets and nine capability controls, assessment lock.
Instructor learner review	Submission inspection, learner detail, concept/course/project progress and reset.
Instructor authoring	Custom lessons and custom guided projects/checkpoints.
Reporting	Learner recap, teacher report, cohort report, CSV export and teacher-mode reports.
Service diagnostics	Optional AI service status, storage backend status and Intel toolkit status.
Professional transition guidance	CodeUp vs VS Code/Codex/Copilot/Quorum, coding pathway and VS Code handoff.

Function family	Included capability
Nonvisual orientation and indentation	Why-is-this-line-indented, enclosing-block chain, current indentation depth, overview/semantic skim, parent/child/sibling block navigation, Mental Map, and textual program flow.
Precision reading and semantic find	Exact-line reading, punctuation-only reading, token spelling, character-by-character reading, exact identifier comparison, and role-labelled whole-word search with lexical fallback on broken syntax.
Context resume and progressive disclosure	Back-to-code/where-was-I focus restoration plus more/details/exact follow-ups that reject stale explanation context after code changes.
Braille compact view	Experimental D-depth indentation compaction for Braille displays, with explicit on/off commands.

Part V. Repository Feature Coverage

Appendix

This appendix maps the first-party repository surfaces reviewed for this guide so technical and accessibility reviewers can connect user-facing workflows to implementation. This edition was re-audited against main at commit a45665f (7 September 2026), with the guide refresh completed on 10 September 2026. The audit prioritized every first-party user-facing module, command grammar, route/template surface, runtime/accessibility feature, and regression-test family. Third-party vendored Monaco and React distributions remain dependencies and are excluded from CodeUp feature narration.

87. Root application and configuration

Path	Operational role
<u>app.py</u>	Primary Flask application and main IDE routes/command orchestration. Large integration point connecting runtime, commands, accessibility, learning, projects, and classroom.
<u>README.md</u>	Public product overview, modes and representative command reference.
<u>.env.example</u>	Environment-variable examples. Never distribute real secrets.
<u>Procfile</u>	Hosted process start configuration.
<u>requirements.txt</u>	Core Python dependencies.
<u>requirements-dev.txt</u>	Development/test dependencies.
<u>requirements-intel.txt</u>	Optional Intel demo dependencies; not required for normal classroom use.
<u>SECURITY.md</u>	Security guidance for the project.
package.json / package-lock.json	Front-end tooling/dependency metadata.
<u>pytest.ini</u>	Test-suite configuration.
<u>docs/CLASSROOM PRODUCTION STORAGE.md</u>	Production classroom storage, migration, and deployment guidance.
<u>scripts/migrate classroom sqlite to postgres.py</u>	SQLite-to-PostgreSQL classroom migration utility with dry-run and validation support.

88. Accessibility package coverage

Module	Behavior it supports
<u>codeup/accessibility/audio_blocks.py</u>	Audio Blocks creation, navigation, compilation and accessible block interaction.
<u>codeup/accessibility/audio_diff.py</u>	Before/after change review and audio-oriented diff explanation.
<u>codeup/accessibility/error_replay.py</u>	Recent error/mistake replay behavior.
<u>codeup/accessibility/landmarks.py</u>	Accessible navigation/landmark-oriented support.
<u>codeup/accessibility/screen_reader_bridge.py</u>	Screen-reader-aware formatting/bridging behavior.
<u>codeup/accessibility/speech_output.py</u>	Speech-output helpers used by accessible narration.
<code>codeup/accessibility/precision_reader.py</code>	Precision Mode: exact line reading, punctuation reading, token spelling, character-by-character reading, and exact comparisons.

89. Classroom package coverage

Module	Behavior it supports
<u>codeup/classroom/ai_policy.py</u>	Nine assignment capabilities, presets and accessibility exclusions.
<u>codeup/classroom/concepts.py</u>	Concept evidence/progress state.
<u>codeup/classroom/curriculum.py</u>	12-module Python Foundations curriculum and deterministic attempt/challenge checks.
<u>codeup/classroom/db.py</u>	Backend-neutral classroom data-access API for instructors, cohorts, learners, assignments, progress, help, curriculum, guided projects, concepts, and activity events.
<u>codeup/classroom/guided_projects.py</u>	Built-in and custom guided-project checkpoints.
<u>codeup/classroom/ide_commands.py</u>	Deterministic classroom commands, navigation, join, assignments, course, projects, and help.
<u>codeup/classroom/learner_actions.py</u>	Learner-side classroom actions such as join, submit, and help.
<u>codeup/classroom/learner_context.py</u>	Current learner/cohort/assignment context for the IDE.

Module	Behavior it supports
<u>codeup/classroom/reports.py</u>	Cohort/learner reporting logic.
<u>codeup/classroom/routes.py</u>	Instructor and learner HTTP routes, authentication, dashboards, assignments, help queue, reports, curriculum and classroom APIs.
<u>codeup/classroom/ storage.py</u>	Classroom storage backend selection, SQLite compatibility, PostgreSQL connection pooling, transactions, and backend health.
<u>codeup/classroom/ postgres schema.py</u>	PostgreSQL classroom schema, constraints, indexes, and versioned initialization.

90. Command-system coverage

Module	Behavior it supports
<u>codeup/commands/beginner templates.py</u>	Beginner program templates and common program-generation patterns.
<u>codeup/commands/clarification flow.py</u>	Clarification flow when a command needs more information.
<u>codeup/commands/command clarifier.py</u>	Clarifies ambiguous command intent.
<u>codeup/commands/command normalization.py</u>	Normalizes phrasing before routing.
<u>codeup/commands/ conversation orchestrator.py</u>	Coordinates multi-turn command interactions and follow-ups.
<u>codeup/commands/deterministic code tools.py</u>	Non-AI code analysis/editing tools where deterministic behavior is possible.
<u>codeup/commands/input concierge.py</u>	Python input() prompt/value handling.
<u>codeup/commands/intent parser.py</u>	Large intent-routing surface for CodeUp commands.
<u>codeup/commands/intent repair.py</u>	Repairs/recovers near-miss command interpretations.
<u>codeup/commands/natural code editor.py</u>	Natural-language edits to existing code.
<u>codeup/commands/ natural command mapper.py</u>	Maps natural language onto supported actions.
<u>codeup/commands/symbolic specs.py</u>	Structured specifications for deterministic/symbolic command behavior.

90.1 Command grammar audit

The detailed Section 80 catalog is derived from the current intent parser plus first-party routing and regression tests. It documents canonical user-facing phrases rather than every duplicate alias, Hindi/Hinglish repair phrase, speech-recognition near miss, or internal parser regex. This keeps the guide usable while still covering every command function family exposed by the current command map.

91. Learning package coverage

Module	Behavior it supports
<u>codeup/learning/accessible_learning.py</u>	Accessible learning packs and lesson-oriented workflows.
<u>codeup/learning/ask_code.py</u>	Questions about the learner's current code.
<u>codeup/learning/concept_qa.py</u>	Python concept question answering.
<u>codeup/learning/concept_tutor.py</u>	Tutor-oriented concept guidance.
<u>codeup/learning/hint_engine.py</u>	Progressive hints.
<u>codeup/learning/learning_moat.py</u>	Learning-oriented behavior designed to keep the learner engaged in reasoning.
<u>codeup/learning/learning_recap.py</u>	Learner recap generation.
<u>codeup/learning/lesson_builder.py</u>	Lesson construction/support.
<u>codeup/learning/literacy_mode.py</u>	Programming Literacy Mode missions and understanding checks.
<u>codeup/learning/trainer_review.py</u>	Trainer/instructor review support.
<u>codeup/learning/tutorial_engine.py</u>	Guided onboarding/tutorial state and actions.

92. Project package coverage

Module	Behavior it supports
<u>codeup/projects/export_support.py</u>	Project export, including handoff/export patterns.
<u>codeup/projects/project_map.py</u>	Project-level structural map.
<u>codeup/projects/project_support.py</u>	Project file lifecycle and supporting operations.
<u>codeup/projects/structure_parser.py</u>	Parses source structure.

Module	Behavior it supports
<u>codeup/projects/structure_tools.py</u>	Cursor-aware indentation/context, hierarchical code maps, overview, relative block navigation, textual program flow, experimental Braille compact view, and semantic find.

93. Runtime package coverage

Module	Behavior it supports
<u>codeup/runtime/debug_teacher.py</u>	Teaching-oriented debugging explanations/walkthrough support.
<u>codeup/runtime/error_trace.py</u>	Runtime error trace analysis and narration inputs.
<u>codeup/runtime/sandbox_runner.py</u>	Sandboxed Python execution.
<u>codeup/runtime/sandboxed_fs.py</u>	Sandboxed file-system behavior for project execution.
<u>codeup/runtime/session_memory.py</u>	Session context/memory for follow-up commands and recent state.
<u>codeup/runtime/state_watch.py</u>	Program-state / variable-watch behavior.

94. AI/integration/provider coverage

Module	Behavior it supports
<u>codeup/integrations/groq_key_manager.py</u>	Optional Groq key/service management.
<u>codeup/integrations/grounded_ai.py</u>	AI assistance grounded in current code/runtime/context.
<u>codeup/providers/groq_pool.py</u>	Provider pooling/reliability behavior for optional Groq AI.
<u>codeup/integrations/intel_showcase.py</u>	Optional Intel demonstration surface. It is not needed for normal classroom use.
<u>codeup/integrations/opencvino_intent_demo.py</u>	Optional OpenVINO intent demo. It is not required for the normal classroom workflow.

95. Reporting package coverage

Module	Behavior it supports
<u>codeup/reports/report_support.py</u>	General report support used by CodeUp learning/project reporting.
<u>codeup/reports/teacher_report.py</u>	Teacher-facing report generation outside/alongside classroom-specific reports.

96. First-party browser JavaScript coverage

File	Behavior it supports
<u>static/app.js</u>	Main IDE interaction, command responses, editor/output state and large portions of browser-side behavior.
<u>static/classroom.js</u>	Learner classroom integration inside /ide, progressive disclosure, assignment/help state and learner live sync.
<u>static/instructor-sync.js</u>	Focus-preserving instructor near-live learner/assignment/help updates and meaningful event announcements.
<u>static/form-error-focus.js</u>	Accessible focus + announcement after server-side instructor form validation errors.
<u>static/live-assistant.js</u>	Live assistant browser interaction.
<u>static/voice-engine.js</u>	Voice recognition/command behavior.
<u>static/tts-detect.js</u>	Browser text-to-speech detection/support.
<u>static/tutorial.js</u>	Tutorial/onboarding browser workflow.
<u>static/python.worker.js</u>	Browser worker support used by Python-related browser processing.
static/landing/*.jsx + dist/bundle.js	Public landing-page presentation. Does not define the core classroom learning model.

97. User-facing templates coverage

Template	Operational surface
<u>templates/index.html</u>	Main IDE: settings, editor, output, commands, learning tools, Audio Blocks and classroom integration.

Template	Operational surface
<u>templates/accessibility.html</u>	Accessibility information/help surface.
<u>templates/accessible_coding_tools.html</u>	Accessible coding-tool presentation/help.
<u>templates/landing.html</u>	Landing page shell.
<u>templates/classroom/base.html</u>	Classroom semantic shell, skip link, centralized status/alert regions.
<u>templates/classroom/login.html</u>	Instructor sign-in/registration and learner join link.
<u>templates/classroom/instructor_dashboard.html</u>	Create/manage cohorts.
<u>templates/classroom/cohort_dashboard.html</u>	Cohort metrics, learners, assignments and cohort controls.
<u>templates/classroom/assignment_detail.html</u>	Publishing, schedule, permissions, assessment lock and submissions.
<u>templates/classroom/learner_home.html</u>	Learner assignments, course, projects, and help.
<u>templates/classroom/learner_detail.html</u>	Instructor learner progress/report view.
<u>templates/classroom/help_queue.html</u>	Open, helping, and resolved instructor help workflow.
<u>templates/classroom/custom_lessons.html</u>	Instructor-authored lessons.
<u>templates/classroom/custom_projects.html</u>	Instructor-authored guided projects/checkpoints.
<u>templates/classroom/curriculum_home.html</u>	Learner course/module navigation.
<u>templates/classroom/quiz.html</u>	Curriculum quiz interaction.
<u>templates/classroom/report.html</u>	Cohort report page.
<u>templates/classroom/submission_view.html</u>	Submission inspection.
<u>templates/classroom/join.html</u>	Fallback learner cohort join.
<u>templates/classroom/confirm.html</u>	Confirmation page for classroom actions requiring confirmation.
<u>templates/classroom/groq_status.html</u>	Authenticated AI-service status diagnostic.
<u>templates/classroom/storage_status.html</u>	Authenticated classroom storage backend, health, and schema-version diagnostic.

98. Styles and front-end accessibility assets

Path	Role
<u>static/style/core.css</u>	Core IDE visual layout/styles.
<u>static/style/accessibility.css</u>	Accessibility-specific visual helpers.
<u>static/style/ui-improvements.css</u>	UI presentation refinements.
<u>static/style/classroom.css</u>	Classroom page styles and accessible error presentation.
<u>static/style/fonts.css</u>	Bundled font declarations.
<u>static/vendor/fonts/*</u>	Bundled fonts, including Atkinson Hyperlegible and JetBrains Mono.
<u>static/vendor/monaco/*</u>	Third-party Monaco Editor distribution. Dependency, not individually documented as CodeUp logic.
<u>static/vendor/react/*</u>	Third-party React distribution for landing-page components. Dependency, not individually documented as CodeUp logic.

99. Built-in lesson files and documentation

Path	Role
<u>lessons/lesson_1_print.md</u>	Early print/output learning material.
<u>lessons/lesson_2_variables.md</u>	Variable learning material.
<u>lessons/lesson_3_loops.md</u>	Loop learning material.
<u>docs/nvda-review-fixes.md</u>	Engineering record of screen-reader/NVDA-oriented review fixes and accessibility improvements.

100. Test coverage map

The repository contains a large Python and JavaScript regression suite. The test surfaces below are grouped by the behavior they protect, which lets technical reviewers see where major workflows have regression coverage.

QA group	Representative test files / protected behavior
Classroom core	test_classroom_db.py, test_classroom_acceptance_flow.py, test_classroom_hardening.py, test_classroom_reports.py, test_classroom_curriculum.py, test_classroom_guided_projects.py, test_classroom_storage_backend.py, test_classroom_postgres_acceptance.py, test_migrate_classroom_sqlite_to_postgres.py
Classroom IDE/commands	test_classroom_ide_commands.py, test_classroom_ide_integration.py, test_classroom_join_robustness.py, test_classroom_demo_fixes.py, test_classroom_live_sync.py
Instructor accessibility/live sync	test_classroom_accessibility.py, test_classroom_form_validation.py, test_instructor_sync_hardening.py, test_ide_accessibility_semantics.py, test_nvda_review_fixes.py
Audio/accessibility	test_audio_blocks_mode.py, test_audio_diff.py, test_audio_features.py, test_accessibility_speech_contract.py, test_screen_reader_bridge.py, test_voice_accessibility_regressions.py, test_indentation_moat.py, test_efficiency_pass.py, test_accessibility_pass_2.py, test_xrcvc_v2_remediation.py, test_xrcvc_v2_closure_pass.py, test_accessibility_axe.py, test_monaco_tab_focus.py
Runtime/debugging/input	test_error_trace.py, test_error_replay.py, test_state_watch.py, test_python_input_flow.py, test_run_output_speech.py
Commands/editing	test_deterministic_code_tools.py, test_deterministic_editing_tools.py, test_natural_code_editor.py, test_natural_command_mapper.py, test_intent_repair.py, test_routing_safety.py
Learning	test_accessible_learning_packs.py, test_programming_literacy_mode.py, test_concept_questions.py, test_concept_tutor.py, test_hint_engine.py, test_tutorial_engine.py
Projects/export	test_multifile_projects.py, test_project_map.py, test_project_report.py, test_export_project.py
Optional AI/provider	test_grounded_ai.py, test_groq_pool.py, test_groq_integration_concurrency.py, test_groq_key_manager.py

QA group	Representative test files / protected behavior
Browser/voice JavaScript	live_assistant.test.js, spoken_code.test.js, tutorial_model.test.js, voice_speech_chunking.test.js, announcement_ownership.test.js, plus Python wrappers that assert speech-mode, focus, and front-end accessibility contracts.

101. Deterministic features versus optional AI

Mostly deterministic / local logic	Optional AI-assisted
Python execution and output	Open-ended code generation
Classroom join/navigation/submission/help commands	Some natural-language explanations
Instructor cohort/assignment records	Some fix/explanation/tutor responses
Curriculum attempt/challenge checks	Open-ended concept tutoring where configured
Guided-project checkpoints	Grounded AI assistance where a provider is configured
Code/project structure parsing	AI may explain parsed structure in natural language when enabled.
Many editing/navigation commands	AI may assist with open-ended editing requests when enabled.
Accessibility semantics and keyboard navigation	No AI required.

This separation lets instructors choose AI assistance according to the learning goal while keeping the core classroom workflow consistent.

102. Program launch checklist

Check	Pass condition
Deployment	Live CodeUp opens normally and the instructor and learner entry points are reachable.
Instructor	Instructor account created and sign-in tested.
Cohort	Cohort created and join code tested from a learner browser.
Learner devices	Browser and usual assistive technology checked on representative learner machines.

Check	Pass condition
Assignment	One sample assignment published and opened from a learner profile.
Help	Learner help request appears in instructor Help queue.
Submission	Learner submission appears and can be inspected.
Accessibility	Representative learner devices, browsers, keyboards, and usual assistive technology are ready for class.
Keyboard path	Learners and instructors know the typed-command and keyboard workflow in addition to optional voice.
Support	A clear support contact is available for product issues during classroom use.
Input syntax	Learners are taught the canonical value form use <code><value></code> , for example use Alexis or use 16, and the guide does not instruct them to append “as input”.

103. Suggested message to learners before the first session

“CodeUp is a Python learning environment built around keyboard, screen-reader, and optional voice workflows. You can type commands or use voice whenever you prefer. During class, please tell your instructor whenever navigation, speech, instructions, or coding feedback could be clearer so we can support you immediately.”